

SOC Windows Event Log Investigation

Case Study — CASE-002

Analyst: Satya Upendra Samana | **Date:** July 2026 | **Environment:** Personal Windows Laptop

1. Background — How This Started

In July 2026, I was working through a self-directed SOC L1 learning path — starting from absolute OS fundamentals, moving through Windows Security event log concepts, attack pattern recognition, and log correlation. By the time I reached the final project phase, I had studied all the core Event IDs, brute force patterns, privilege escalation techniques, and persistence mechanisms. But I had never actually sat down, generated a real attack chain on my own machine, and investigated the resulting logs the same way a real SOC analyst would.

That gap bothered me. Reading about events in theory is very different from looking at raw log entries on a machine you control and building a full investigation story out of them. So, I decided to close that gap: simulate the exact attack sequence I had been studying — brute force, initial access, backdoor account creation, privilege escalation, service-based persistence — safely, on my own personal laptop, with no external network involved, and then investigate every step as if it were a live incident.

This is the investigation report for that exercise. CASE-002 follows the same format as my earlier phishing analysis (CASE-001) because they are part of the same portfolio series — different attack types, same analyst, same investigation methodology.

2. Initial Triage

Before touching any PowerShell command, I checked what the machine was configured to log. This is the equivalent of verifying whether CCTV was recording before you look for footage — there is no point running a simulation if key events are silently dropped before they reach the log.

Two things came back immediately from this pre-check that changed the plan:

- The account lockout threshold was 10 failed attempts, not 5 as I had assumed. A brute-force simulation that stopped at 5 failures would never trigger a lockout event (4740) on this machine at all.
- The “Security System Extension” audit subcategory was set to No Auditing by default — meaning Event ID 4697 (service installed) would be silently dropped regardless of what happened on the machine.

Both gaps had to be understood and corrected before the investigation could produce complete evidence. This itself is a real, generalizable finding: default Windows audit policy does not capture every security-relevant event out of the box. A SOC team that has never verified its audit configuration may be missing critical evidence without knowing it.

3. Investigation Steps

Step 1 — Setting Up the Test Environment

The first thing I did was create a separate, controlled test account so that the entire simulation stayed isolated from my real user account (shash) and the built-in Administrator.

```
net user testuser1 Test@1234 /add
```

Action	Outcome
Created test account “testuser1”	Confirmed visible in net user output
Checked lockout policy via net accounts	Lockout threshold: 10 attempts, duration: 10 minutes
Checked audit policy via auditpol	Security System Extension: No Auditing — gap identified, corrected before simulation

What this led to next: With the environment properly configured and the audit gaps corrected, the machine was ready to generate real, reliable evidence. The first stage of the simulation was the brute-force logon attempt.

Step 2 — Simulating the Brute-Force Logon Attempt

I locked the screen, switched to the testuser1 account, and entered wrong passwords 9 times in a row, then logged in correctly on the 10th attempt. During this, Windows displayed a “delaying next attempt” message after the 6th failure. That is login throttling — a separate built-in defense layer that progressively slows down retry attempts before the full lockout threshold is even reached. It slowed the attack down but did not stop it.

Event ID	Count	Time Range	Finding
4625	9 failed attempts	14:39:36 – 14:42:11	9 failures in ~2.5 minutes. Speed and volume consistent with automated brute force, not a human mistyping.
4624	1 success	14:42:32	Logon Type 2 (interactive), Source 127.0.0.1. Successful logon 21 seconds after the last failure — initial access achieved.

What this led to next: With initial access simulated and logged, the next move was to act the way an attacker would after getting in: plant a hidden account that can be used to return later, even if the original compromised account’s password gets changed.

Step 3 — Creating the Backdoor Account

In a real post-compromise scenario, the first thing an attacker does is create a secondary account with a name that blends into the system — something that looks like a legitimate service account at a quick glance. I did the same.

```
net user backdoor_svc P@ssw0rd123 /add
```

Field	Finding
Event ID	4720 — A user account was created

Field	Finding
New Account	backdoor_svc
Subject (Creator)	shash — stands in for the attacker's already-compromised session in this simulation
Time	15:30:45
Why this name	Deliberately mimics a legitimate service account to avoid drawing attention during a quick user list review

What this led to next: Creating an account alone is not enough — a standard unprivileged account has very limited power. The next step was to elevate it so the attacker has full control of the machine going forward.

Step 4 — Privilege Escalation

Adding the backdoor account to the built-in Administrators group is the moment the account goes from harmless to dangerous. This is the single most critical point in the entire chain.

```
net localgroup Administrators backdoor_svc /add
```

Field	Finding
Event ID	4732 — A member was added to a security-enabled local group
Member Added	backdoor_svc
Group	BUILTIN\Administrators (SID S-1-5-32-544)
Subject	shash
Time	15:34:18 — 3.5 minutes after the account was created
Why this matters	Before this event: backdoor_svc was harmless. After this event: full administrative control over the host.

What this led to next: With an elevated backdoor account now in place, the final stage was installing something that would survive a reboot — a persistence mechanism that keeps the attacker's access alive even if all passwords on the original compromised account are changed.

Step 5 — Installing the Persistence Service

A new service was registered under a name designed to look like a legitimate Windows component. At this point I hit the first real-world problem: the event did not appear in the log.

```
sc.exe create WinUpdateHelper binPath= "C:\Windows\System32\svchost.exe -k netsvcs"
start= demand
```

The service created successfully (confirmed via [SC] CreateService SUCCESS in PowerShell), but Event 4697 was nowhere in the Security log. I ran the auditpol check again and confirmed the root cause: Security System Extension was still showing No Auditing at the subcategory level despite the top-level category appearing enabled. I corrected it:

```
auditpol /set /subcategory:"Security System Extension" /success:enable /failure:enable
```

Deleted and recreated the service. Event 4697 appeared correctly this time.

Field	Finding
Event ID	4697 — A service was installed in the system
Service Name	WinUpdateHelper
Service File	C:\Windows\System32\svchost.exe -k netsvcs
Service Account	LocalSystem — highest privilege level on Windows
Time	15:49:21
Name disguise	Designed to resemble a Windows Update component — a defender quickly scanning service names would not immediately flag it

4. Key Findings

- 9 failed logon attempts (4625) against testuser1 in ~2.5 minutes — speed and volume consistent with automated brute force, not human typing error
- Successful logon (4624) 21 seconds after the final failure — brute force succeeded, initial access achieved
- Windows login throttling activated at attempt 6, slowing but not fully blocking the attack — the account did not reach the 10-attempt lockout threshold
- New local account “backdoor_svc” created (4720) with a name deliberately chosen to blend in with legitimate service accounts
- backdoor_svc added to the built-in Administrators group (4732) — the single highest-risk moment in the entire chain
- New service “WinUpdateHelper” installed as LocalSystem (4697) — persistence established, survives reboots and password changes
- Two audit-policy gaps discovered: Security System Extension was No Auditing (initially suppressed 4697), and lockout threshold was 10 not 5 (affected simulation planning)

5. Verdict

Indicator	Finding	Verdict
9x Event 4625 in 2.5 minutes	Rapid automated-speed failures	 Critical
Event 4624 immediately after failures	Brute-force succeeded — initial access	 Critical
Login throttling activated at attempt 6	Defense triggered, did not fully block	 Partial Defense
Event 4720 — backdoor_svc created	New account with disguised name	 Red Flag
Event 4732 — added to Administrators	Full privilege escalation	 Critical

Indicator	Finding	Verdict
Event 4697 — WinUpdateHelper installed	Persistence via disguised service	 Critical
Security System Extension: No Auditing	Audit gap — initially hid Event 4697	 Gap Found
Logon Type 2, Source 127.0.0.1 throughout	All activity confirmed local — no external threat	 Simulation Context

Classification: Controlled Simulation — High Confidence — Brute Force / Privilege Escalation / Persistence Chain

Attack method: Repeated credential guessing to gain initial access, followed by a disguised backdoor account, privilege escalation to Administrator, and service-based persistence designed to survive reboots.

Most critical intervention point: Event 4732. Before this event, backdoor_svc was a harmless standard account. After this event, it had full administrative control of the host. Catching and reverting this single event would have stopped the most damage — even though the account already existed, it was powerless without this step.

6. MITRE ATT&CK Mapping

Note: I am in active learning on MITRE ATT&CK framework usage. The mappings below were identified during this investigation.

Technique ID	Description
T1110 — Brute Force	Repeated failed logon attempts against a single account until one succeeds
T1136.001 — Local Account Creation	New local account “backdoor_svc” created to maintain access beyond the initial compromise
T1098 — Account Manipulation	Backdoor account added to the Administrators group to gain elevated privileges
T1543.003 — Windows Service	Service “WinUpdateHelper” installed to maintain persistence across reboots
T1036 — Masquerading	Service and account names deliberately chosen to resemble legitimate system components

7. What I’d Do Differently in a Real SOC

In this exercise, I was the one who set up the simulation and then investigated my own logs. In a real SOC environment, none of that setup would happen at my level. The alert would arrive from a detection tool or a user report, not from something I triggered myself.

What would be different:

- A SIEM (Splunk / Microsoft Sentinel) would automatically detect the rapid 4625 pattern and raise an alert without me needing to manually filter Event Viewer. Rules like “5+ failed logons against the same account within 3 minutes” are standard across most enterprise SOC environments.
- Event 4732 — adding any account to the Administrators group — would trigger an immediate high-priority automated alert in any properly configured SIEM, not something a junior analyst would find by manually searching.
- Threat intelligence integration (VirusTotal, AlienVault OTX) would cross-reference service names and file paths against known malicious indicators automatically.
- Every finding would be documented inside a ticketing system (ServiceNow / Jira) in real time during the investigation, not written up after the fact.
- Final verdict and containment actions would go through a senior L2 analyst for sign-off, not closed independently by an L1.

Containment actions I would raise to L2:

- Remove backdoor_svc from the Administrators group immediately and disable or delete the account.
- Delete the WinUpdateHelper service.
- Reset testuser1’s credentials and review any other machines the Logon ID appeared on (if domain-joined).
- Audit the Security System Extension and Account Lockout subcategory settings across all endpoints in the environment — the gaps found here may exist on other machines and be silently hiding evidence right now.

Current gap: The investigation techniques and log-reading skills developed here transfer directly to a real SOC environment. What I have not yet practiced is receiving and working alerts inside a live SIEM interface and using a formal ticketing system for escalation. Both are in my active learning roadmap.

8. Tools Used

Tool	Purpose
Windows Event Viewer	Read and filter Security log entries (4625, 4624, 4720, 4732, 4697)
PowerShell — net user / net localgroup	Create and delete test accounts, manage group membership
PowerShell — sc.exe	Create and delete the test Windows service (WinUpdateHelper)
PowerShell — auditpol	Check and correct audit policy configuration before and during the simulation
PowerShell — net accounts	Verify lockout threshold and policy settings before the simulation began
MITRE ATT&CK Framework	Threat technique mapping and attack classification

9. References

- MITRE ATT&CK Framework — <https://attack.mitre.org>
- T1110 Brute Force — <https://attack.mitre.org/techniques/T1110>

- T1136 Create Account — <https://attack.mitre.org/techniques/T1136>
- T1098 Account Manipulation — <https://attack.mitre.org/techniques/T1098>
- T1543 Create or Modify System Process — <https://attack.mitre.org/techniques/T1543>
- T1036 Masquerading — <https://attack.mitre.org/techniques/T1036>
- GitHub Repository — <https://github.com/satyaupendrasamana/windows-event-log-analysis>

Report prepared by: Satya Upendra Samana | SOC Analyst Fresher | Google Cybersecurity Certified |
July 2026